

Richard Smith (richard@metafoo.co.uk)
Andrew Sutton (andrew.n.sutton@gmail.com)
Daveed Vandevoorde (daveed@edg.com)

constexpr! functions

Introduction and motivation

The `constexpr` specifier applied to a function or member function indicates that a call to that function might be valid in a context requiring a constant-expression. It does not *require* that every such call be a constant-expression. Sometimes, however, we want to express that a function should *always* produce a constant when called (directly or indirectly), and a non-constant result should produce an error. Such a function is called an *immediate* function. We propose the *decl-specifier* `constexpr!` for that purpose:

```
constexpr! int sqr(int n) {  
    return n*n;  
}  
constexpr int r = sqr(100); // Okay.  
int x = 100;  
int r2 = sqr(x); // Error: Call does not produce a constant.
```

There is one exception to the rule that a call to a `constexpr!` function must be a constant-expression: If the call appears directly in another `constexpr!`, it need not be a constant-expression at that point (since a call to the enclosing function would ultimately have to produce a core constant expression, we do not have to worry about ever having to evaluate that nested call at run time). This permits composition of `constexpr!` functions:

```
constexpr! int sqrsqr(int n) {  
    return sqr(sqr(n)); // Not a constant-expression at this point,  
}                        // but that's okay.  
  
constexpr int dblsqr(int n) {  
    return 2*sqr(n); // Error: Enclosing function is not  
}                  // constexpr!.
```

Finally, as is the case with a bound member function (e.g., `"x.f"`), a `constexpr!` function can only be called; no pointer or reference to it can be formed:

```
using Int2Int = int(int);  
Int2Int *pf = sqr; // Error.
```

One consequence of this specification is that an immediate function never needs to be seen by a back end. That means that immediate functions are an alternative to many function-style macros. It also means that, unlike plain `constexpr` functions, `constexpr!` functions are unlikely to show up in symbolic debuggers. That runs into the broader issue that programmers currently have few tools to debug compile-time evaluation: This proposal does not aim at solving that problem in any way.

Source Locations

The "Library Fundamentals v. 2" TS contains a "magic" `source_location` class get to information similar to the `__FILE__` and `__LINE__` macros and the `__func__` variable (see [N4529](#) for the current draft, and [N4129](#) for some design notes). Unfortunately, because the "value" of a `source_location` is frozen at the point

`source_location::current()` is invoked, composing code making use of this magic class is tricky: Typically, a function wanting to track its point of invocation has to add a defaulted parameter as follows:

```
void my_log_function(char const *msg,
                    source_location src_loc
                    = source_location::current()) {
    // ...
}
```

This idiom ensure that the value of the `source_location::current()` invocation is sampled where `my_log_function` is called instead of where it is defined.

Immediate (i.e., `constexpr!`) functions, however, create a clean separation between the compilation process and the `constexpr` evaluation process (see also [P0992](#)). Thus, we can make `source_location::current()` an immediate function, and wrap it as needed in other immediate functions: The value produced will correspond to the source location of the "root" immediate function call. For example:

```
constexpr! src_line() {
    return source_location::current().line();
}

void some_code() {
    std::cout << src_line() << '\n'; // This line number is output.
}
```

Variables

Given that `constexpr!` functions are a useful alternative to function-style macros, it is natural to wonder whether we could make `constexpr!` *variables* a useful alternative to non-function-style macros. We'd call such variables — which would never be seen by a back end — *immediate* variables:

```
constexpr! int RN = 42;
int LN = 42;

int f(int const&);

int r1 = f(LN); // Direct reference binding.
int r2 = f(RN); // Not a direct binding (temporary created).
```

Unfortunately, the exact specification of such a facility raises some difficult questions, some of which relate to the topic of enabling class types for nontype template parameters as proposed in [P0732R1](#). This paper therefore does not pursue that extension at this time.

Notes

The desire for functionality like this comes up regularly. Sometimes, the motivation is to eliminate macros, and sometimes it just falls naturally out of the development process (e.g., helper functions meant to create nontype template argument values).

The impetus for the present paper, however, is the work being done by SG7 in the realm of compile-time reflection. There is now general agreement that future language support for reflection should use `constexpr` functions, but since "reflection functions" typically *have* to be evaluated at compile time, they will in fact likely be *immediate* functions.

The general principles of this facility were first discussed in Kona (2017) as part of the discussion of P0595r0. At the time, the following poll was recorded:

Provide a way to write a function that is guaranteed to be produce a constant expression, violations diagnosed at call site? (Direction poll for [immediate functions], ignoring syntax):
SF: 8 | F: 13 | N: 2 | A: 0 | SA: 0

(See <http://wiki.edg.com/bin/view/Wg21kona2017/P0595R0>. The syntax under consideration at that time was do constexpr instead of constexpr!.)

Wording changes

Add to the grammar of [dcl.spec] paragraph 1:

decl-specifier:
 storage-class-specifier
 defining-type-specifier
 function-specifier
 friend
 typedef
 constexpr
 constexpr!
 inline

Append to [dcl.spec] paragraph 2:

The constexpr and constexpr! shall not both appear in a *decl-specifiers-seq*.

Update [dcl constexpr] paragraph 1 as follows:

The constexprspecifier shall be applied only to the definition of a variable or variable template or the declaration of a function or function template. The constexpr!specifier shall be applied only to the definition of a function or function template. A function or static data member declared with the constexpr or constexpr! specifier is implicitly an inline function or variable (10.1.6). If any declaration of a function or function template has a constexpr or constexpr! specifier, then all its declarations shall contain the constexprthat same specifier. [Note: An explicit specialization can differ from the template declaration with respect to the constexpr or constexpr! specifier. — end note] [Note: Function parameters cannot be declared constexpr. — end note]

Update [dcl constexpr] paragraph 2 as follows:

A constexpr or constexpr! specifier used in the declaration of a function that is not a constructor declares that function to be a *constexpr function*. Similarly, a constexpr or constexpr! specifier used in a constructor declaration declares that constructor to be a *constexpr constructor*. A function or constructor declared with the constexpr! specifier is called an *immediate function*.

Update [dcl constexpr] paragraph 8 as follows:

The constexpr or constexpr! specifier has no effect on the type of a constexpr function or a constexpr constructor.

Add a paragraph after paragraph 2 of [expr.prim.id]:

An *id-expression* that denotes an immediate function (`_dcl constexpr_`) and is not the right hand expression of a class member access operation (`_expr.ref_`) can be used only as the (possibly parenthesized) *postfix-expression* of a function call (`_expr.call_`).

Add a paragraph at the end of [expr.call]:

A call to an immediate function (`_dcl.constexpr_`) that does not lexically appear in the *function-body* (`_dcl.fct.def.general_`) of an immediate function shall be a constant expression (`_expr.const_`).

In bullet (4.3.2) of [expr.ref] paragraph 4, strike

~~The expression can be used only as the left-hand operand of a member function call (12.2.1). [Note: Any redundant set of parentheses surrounding the expression is ignored (8.4). — end note]~~

and add to the end of bullet (4.3) of that same paragraph (after the two sub-bullets):

If `E1.E2` refers to a nonstatic member function (`_class.mfct.non-static_`) or to an immediate function (`_dcl.constexpr_`) the expression can be used only as the left-hand operand of a member function call (12.2.1). [Note: Any redundant set of parentheses surrounding the expression is ignored (8.4). — end note]

Insert a bullet after the first bullet (8.6) of the definition of *needed for constant evaluation* in [expr.const] paragraph 8:

- a `constexpr` function ...
- an immediate function that is named by an expression (`_basic.def.odr_`), or
- a variable ...

and add an example at the end of the paragraph:

[*Example:*

```
struct N {
    N(N const&) = delete;
};
template<typename T> constexpr void bad_assert_copyable() { T t; T t2 = t; }
using ineffective = decltype(bad_assert_copyable());
// bad_assert_copyable is not needed for constant evaluation
// (and thus not instantiated)
template<typename T> constexpr! void assert_copyable() { T t; T t2 = t; }
using check = decltype(assert_copyable());
// error: assert_copyable is instantiated (because it is needed for constant
// evaluation), but the attempt to copy t is ill-formed
```

— *end example*]

Update [dcl.fct.def.default] paragraph 3 as follows:

An explicitly-defaulted function that is not defined as deleted may be declared `constexpr` or `constexpr!` only if it would have been implicitly declared as `constexpr`. [...]

Update [temp.explicit] paragraph 1 as follows:

[...]An explicit instantiation of a function template, member function of a class template, or variable template shall not use the `inline` or `constexpr`, or `constexpr!` specifiers.